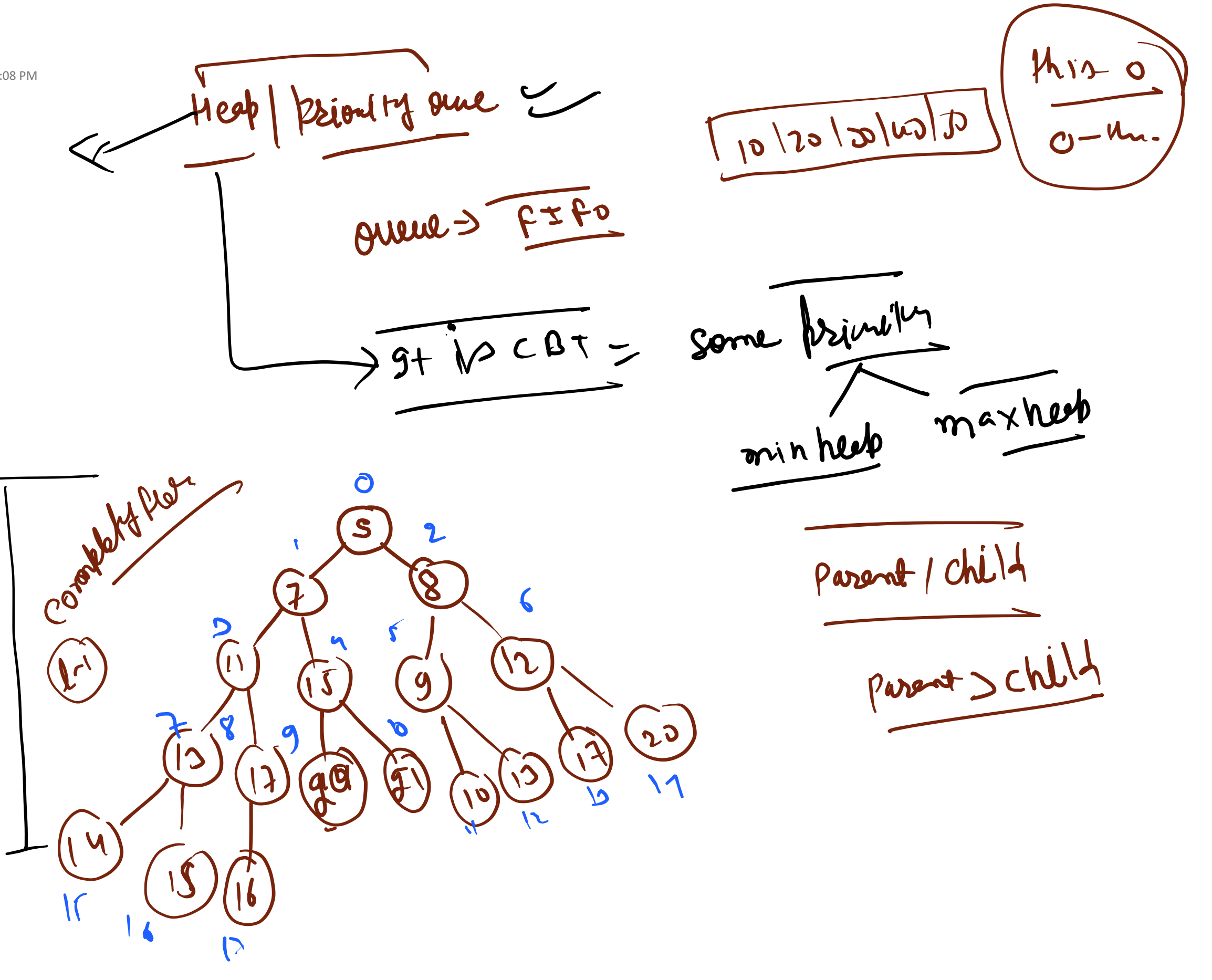
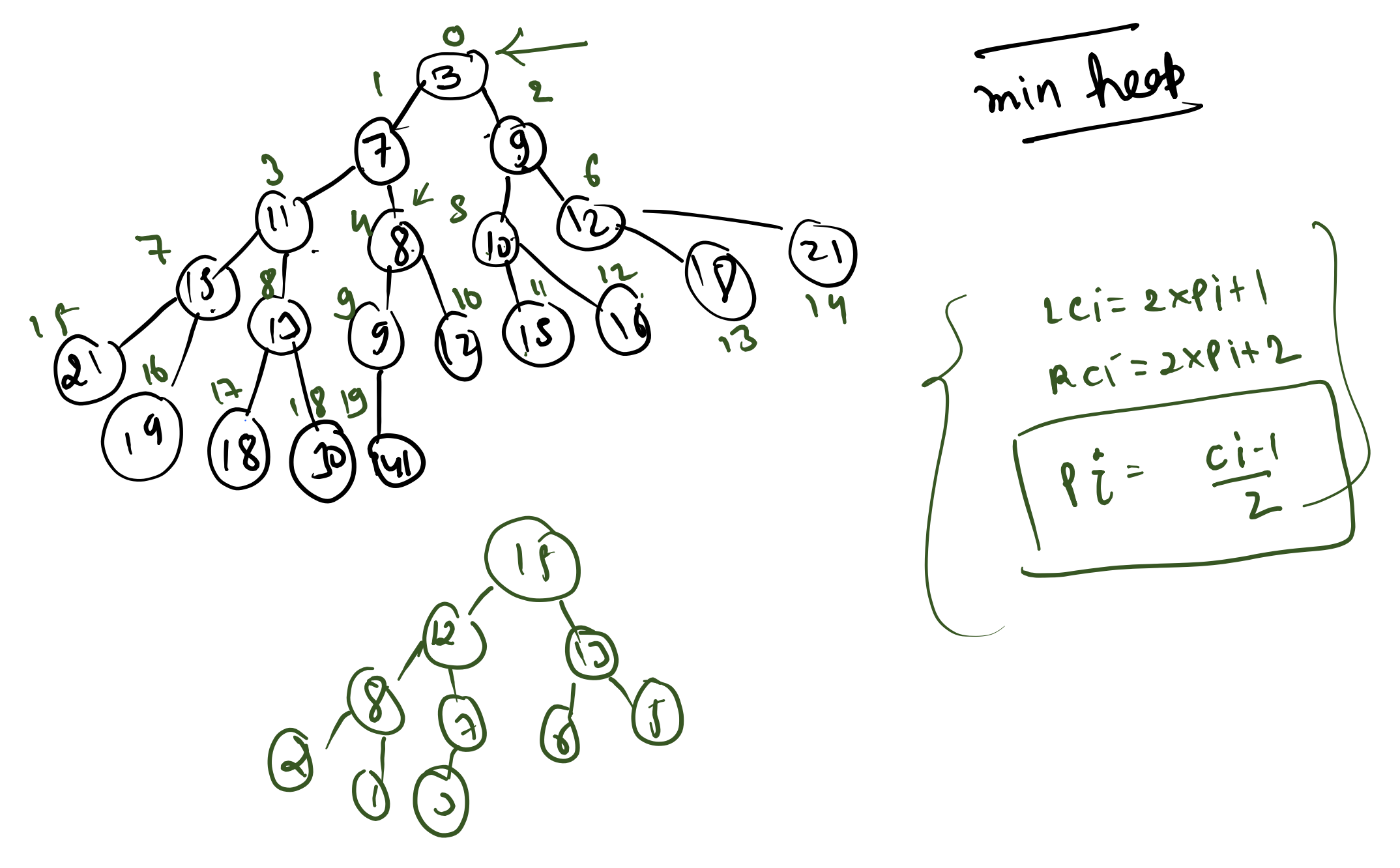
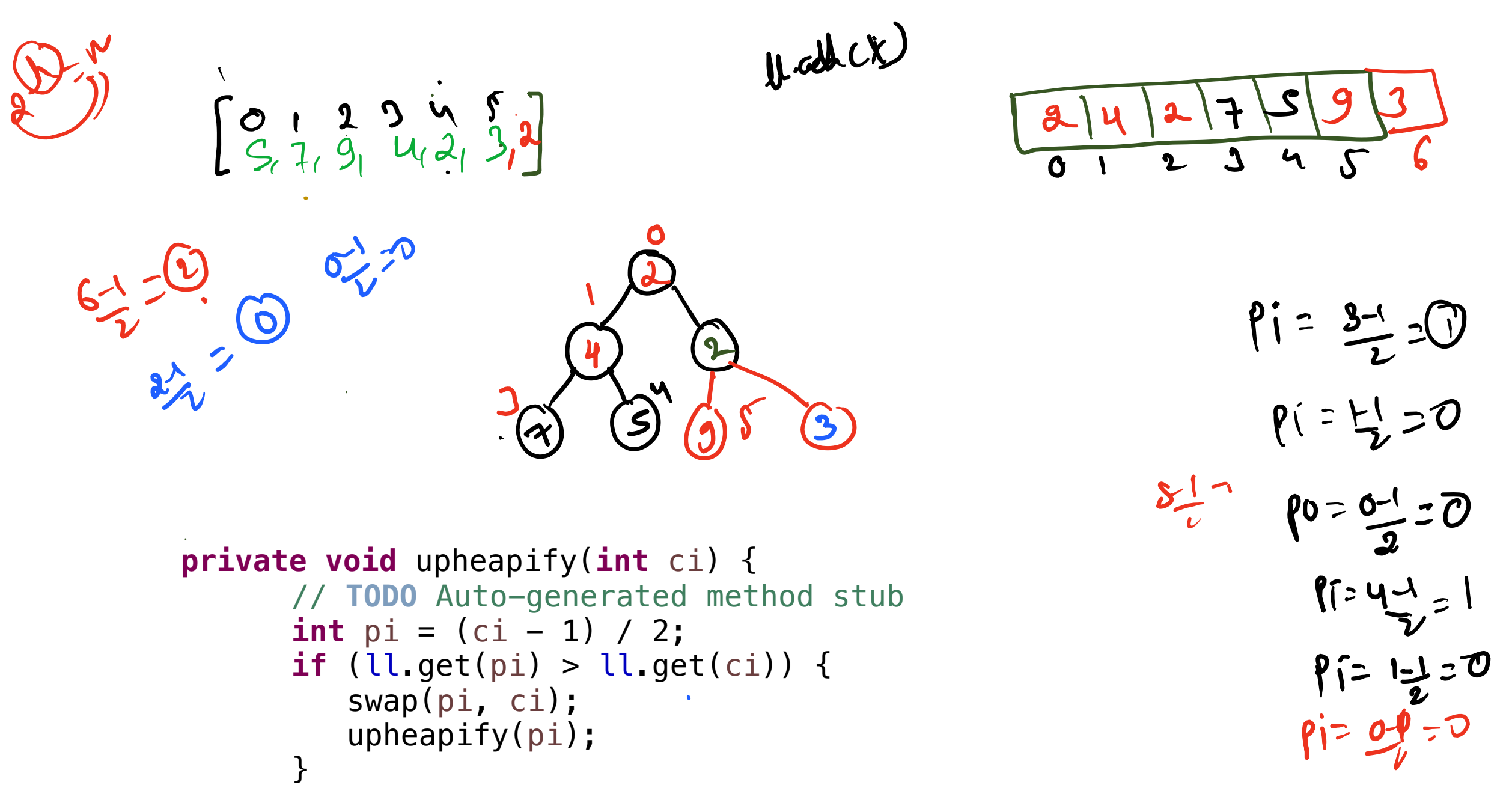
Lec-36

25/10/26 1:08 PM

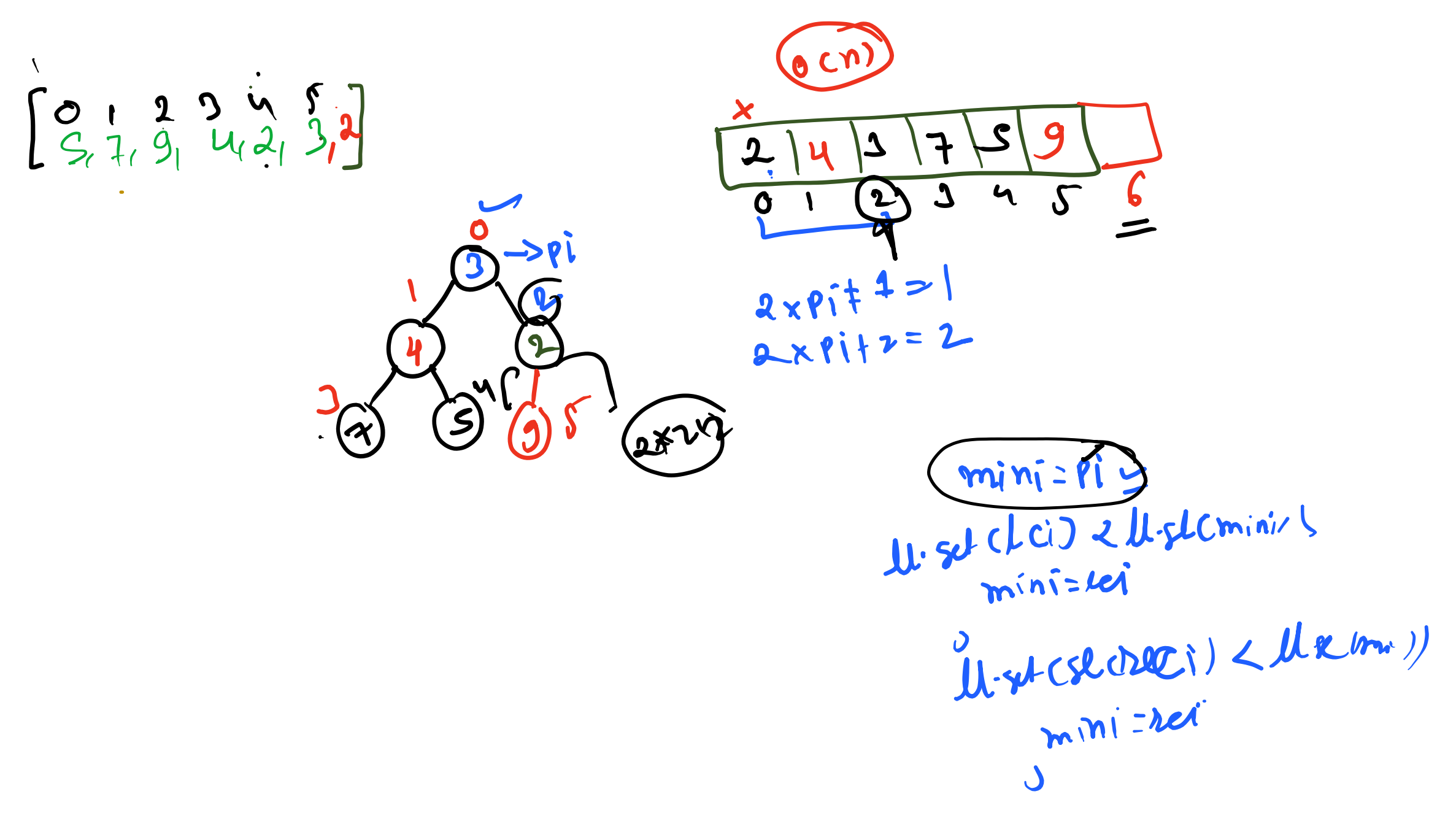


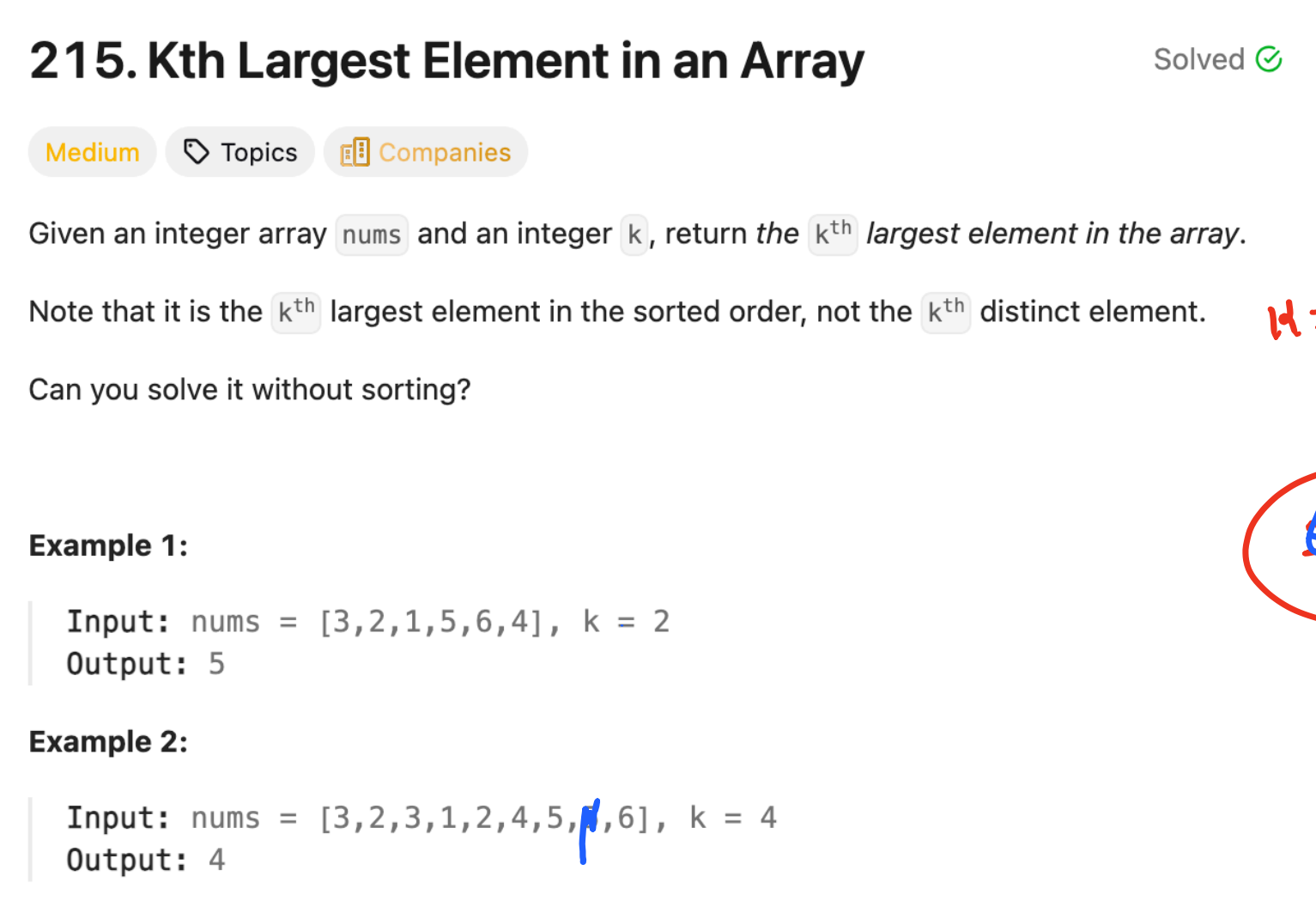


	sorted	unsorted	heap
add	O(n)	O(1)	log(n) -?
remove	O(n)	O(n)	log(n)
get min	O(1)	O(n)	O(1)



```
private void upheapify(int ci) {
        // TODO Auto-generated method stub
        int pi = (ci - 1) / 2;
        if (ll.get(pi) > ll.get(ci)) {
            swap(pi, ci);
            upheapify(pi);
        }
    }
```





215. Kth Largest Element in an Array

Solved ✓

Medium Topics Companies

Given an integer array `nums` and an integer `k`, return *the* `kth` *largest element in the array*.

Note that it is the `kth` largest element in the sorted order, not the `kth` distinct element.

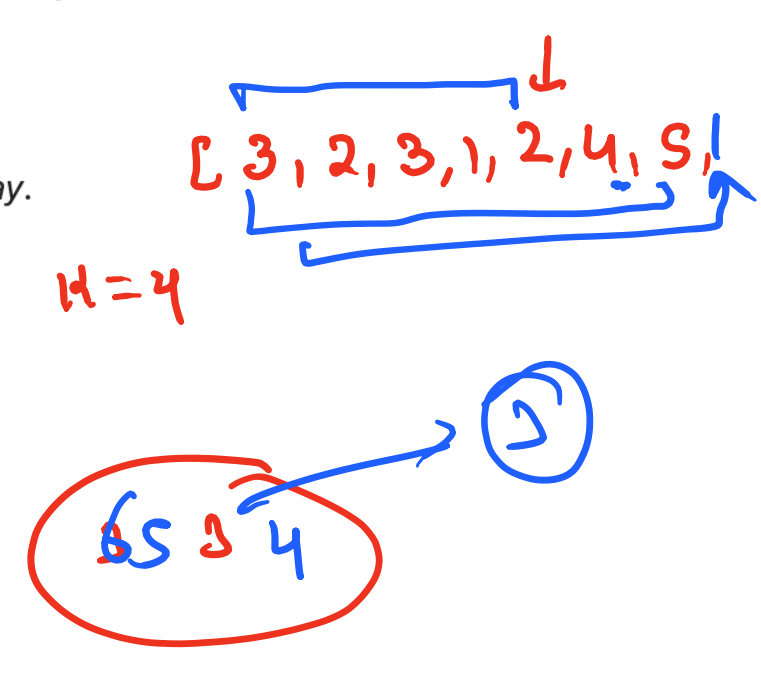
Can you solve it without sorting?

Example 1:

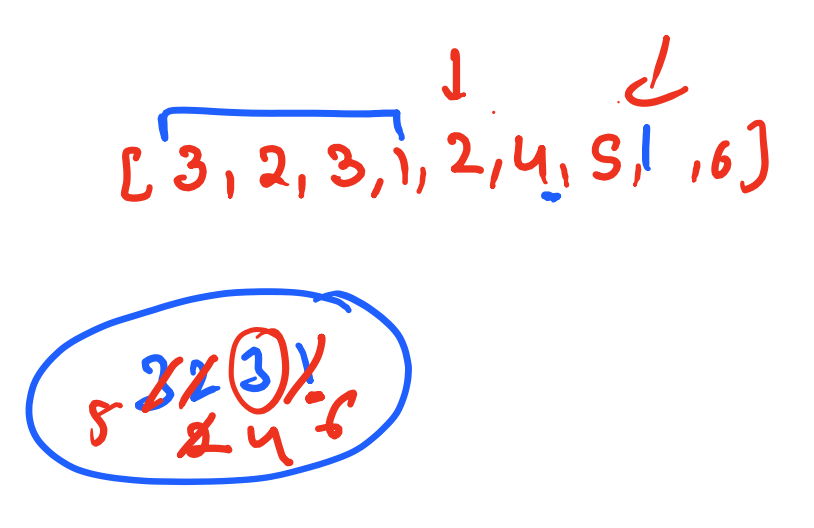
```
Input: nums = [3,2,1,5,6,4], k = 2
Output: 5
```

Example 2:

```
Input: nums = [3,2,3,1,2,4,5,5,6], k = 4
Output: 4
```

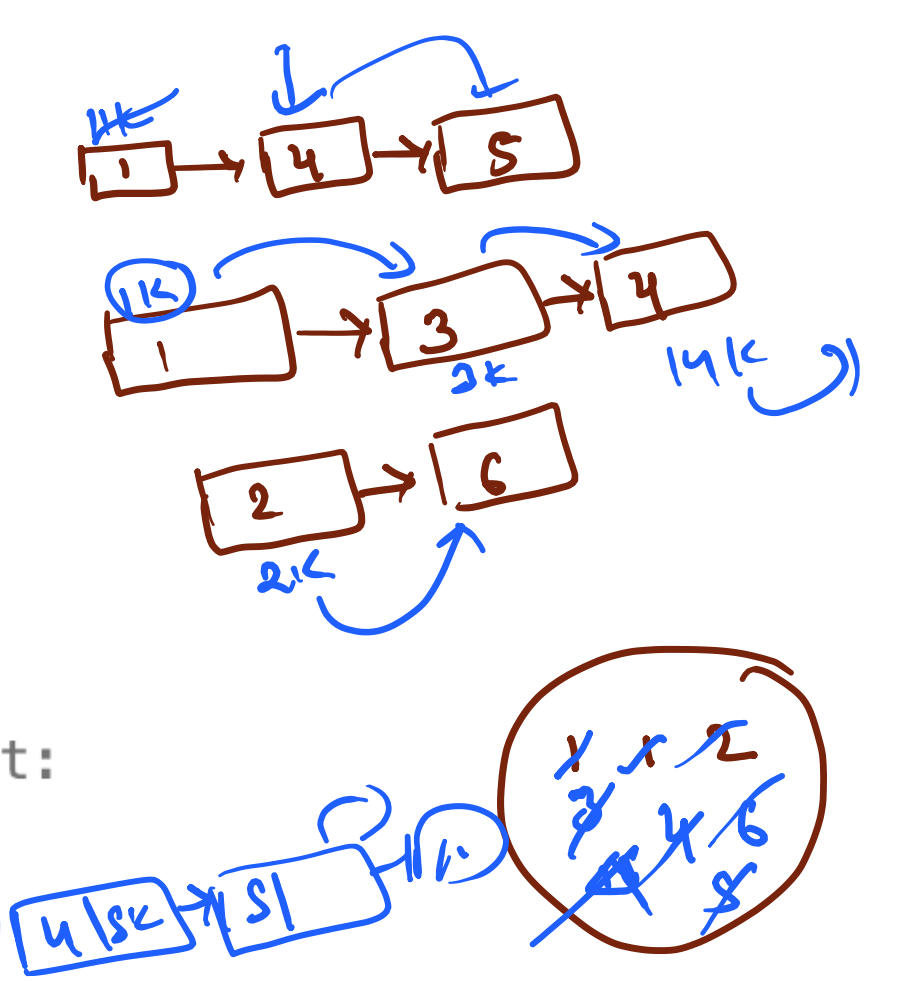


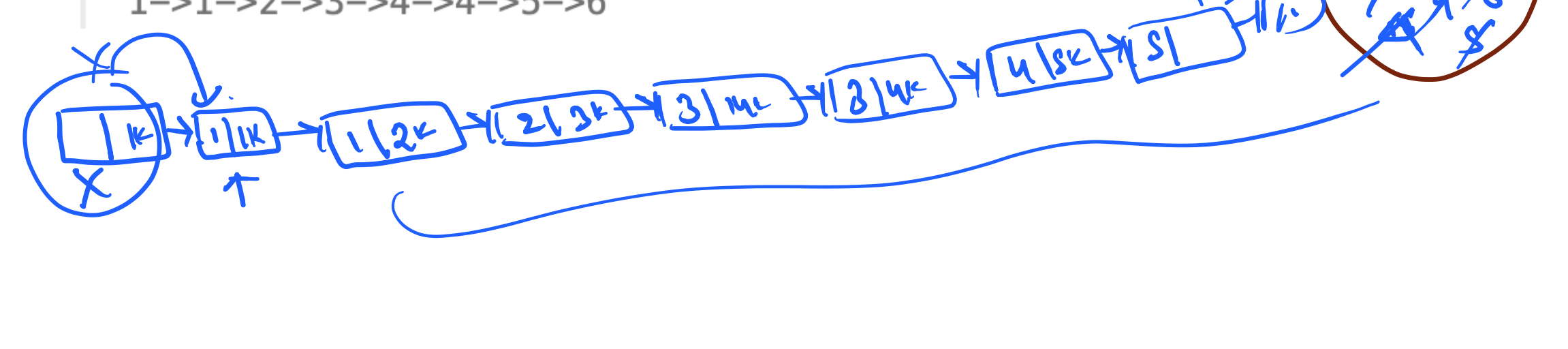
```
public static int Kth_Largest_Element(int[] arr, int k) {
        PriorityQueue<Integer> pq = new PriorityQueue<>();
        for (int i = 0; i < k; i++) {
            pq.add(arr[i]);
        }
        for (int i = k; i < arr.length; i++) {
            if (arr[i] > pq.peek()) {// peek --> view
                pq.poll();
                pq.add(arr[i]);
            }
        }
        return pq.peek();
    }
```



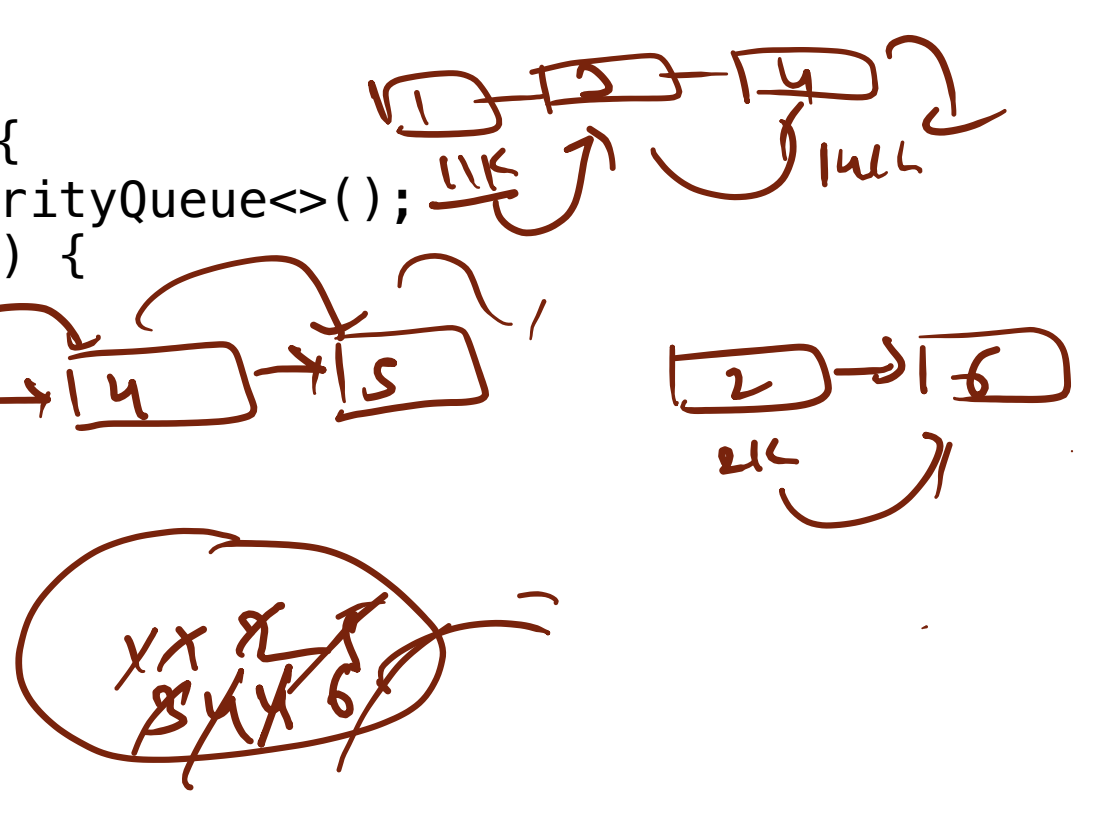
(KN) x log(KN)

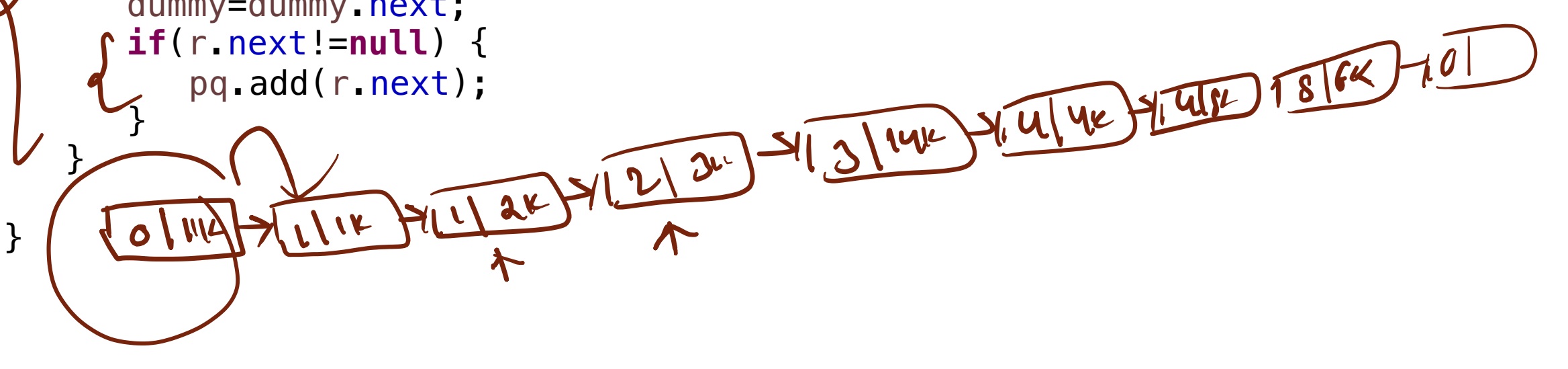
```
Input: lists = [[1,4,5],[1,3,4],[2,6]]
Output: [1,1,2,3,4,4,5,6]
Explanation: The linked-lists are:
[
  1->4->5,
  1->3->4,
  2->6
]
merging them into one sorted linked list:
1->1->2->3->4->4->5->6
```

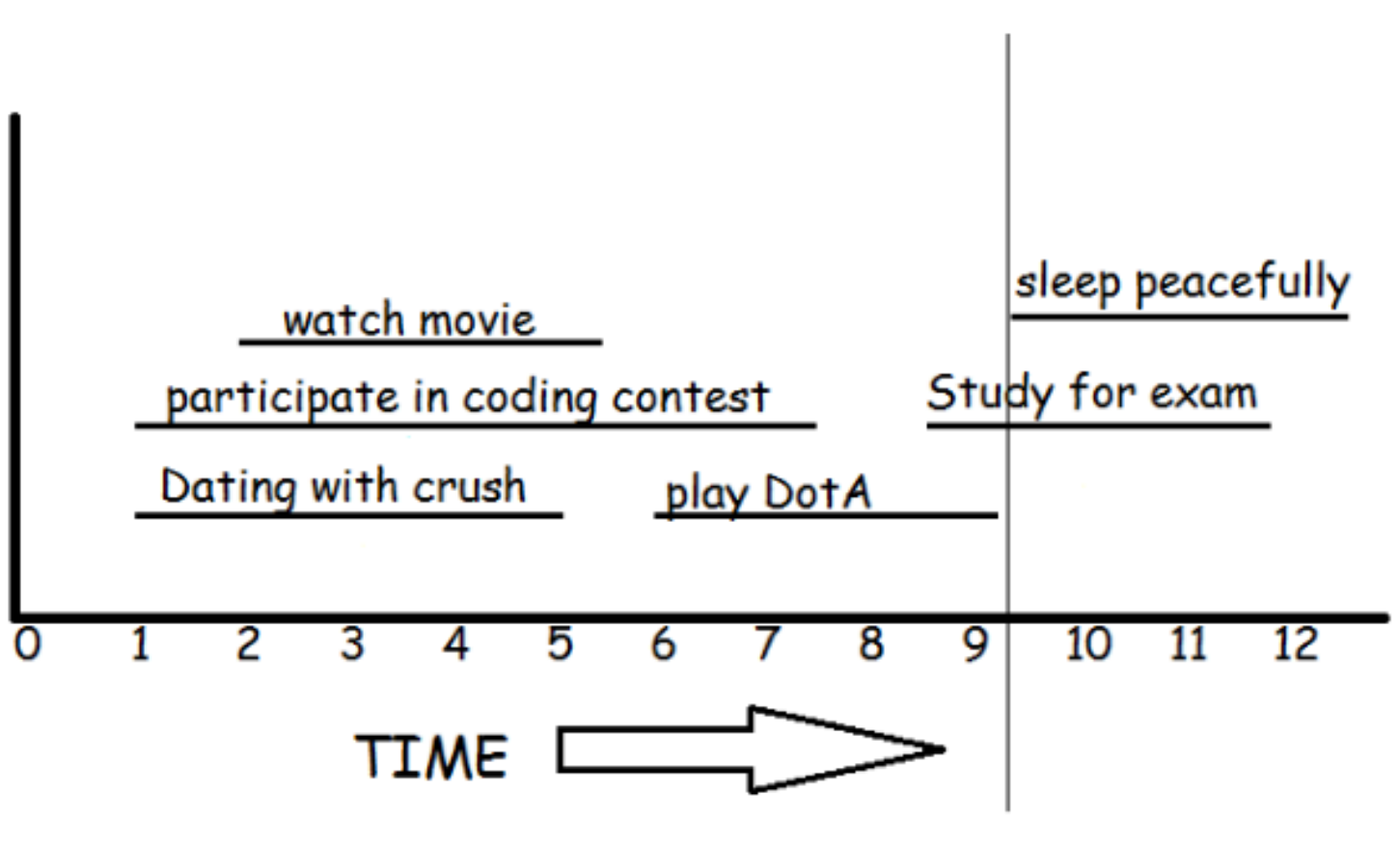




```
public ListNode mergeKLists(ListNode[] lists) {
        PriorityQueue<ListNode> pq = new PriorityQueue<>();
        for (int i = 0; i < lists.length; i++) {
            if (lists[i] != null) {
                pq.add(lists[i]);
            }
        }
        ListNode dummy = new ListNode();
        ListNode temp=dummy;
        while(!pq.isEmpty()) {
            ListNode r=pq.poll();
            dummy.next=r;
            dummy=dummy.next;
            if(r.next!=null) {
                pq.add(r.next);
            }
        }
    }
```

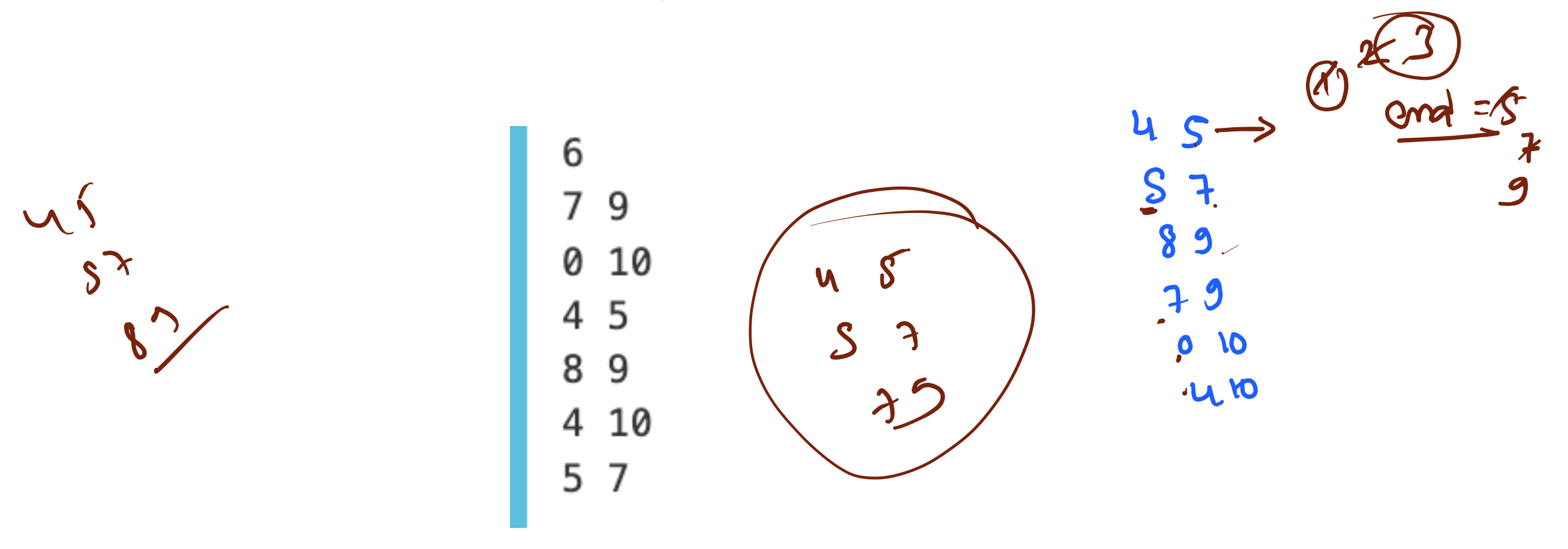


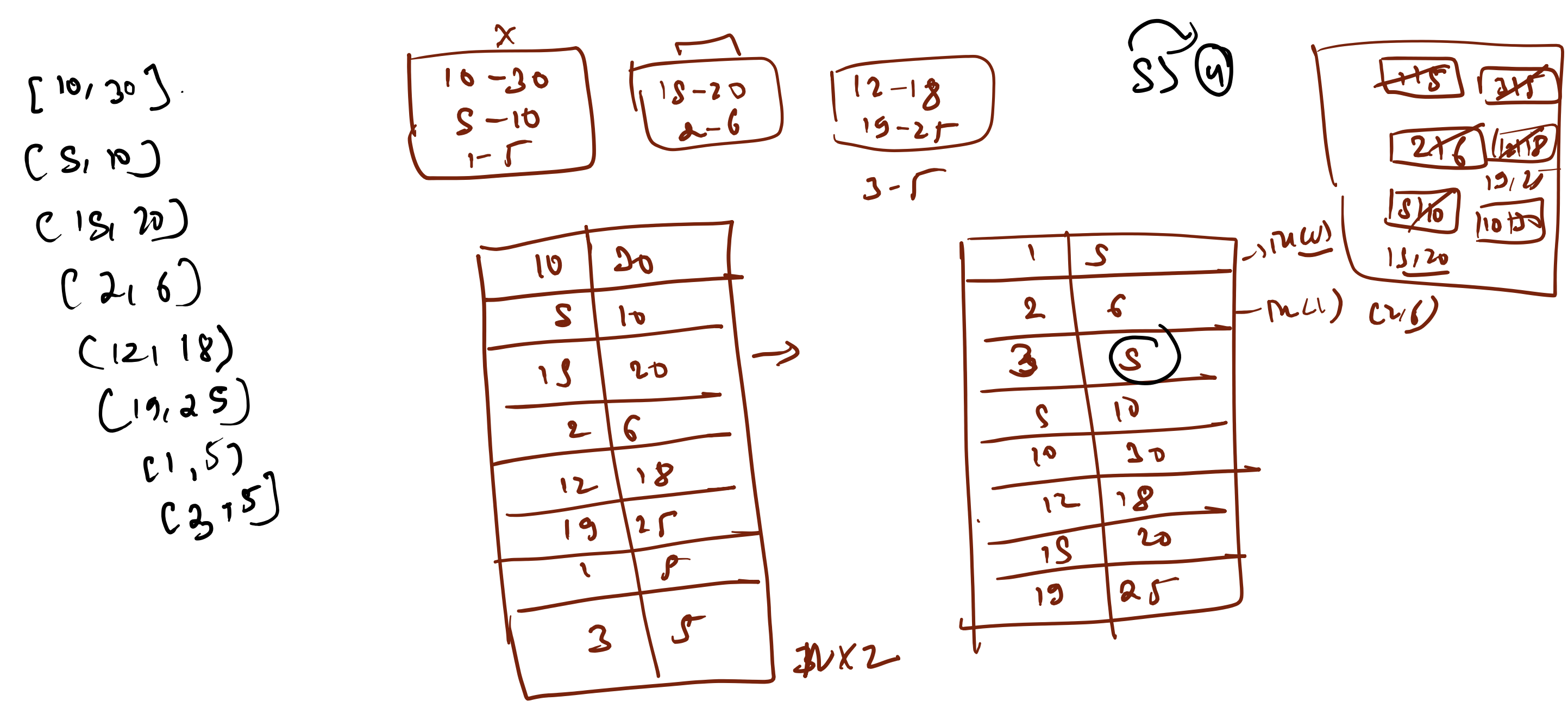


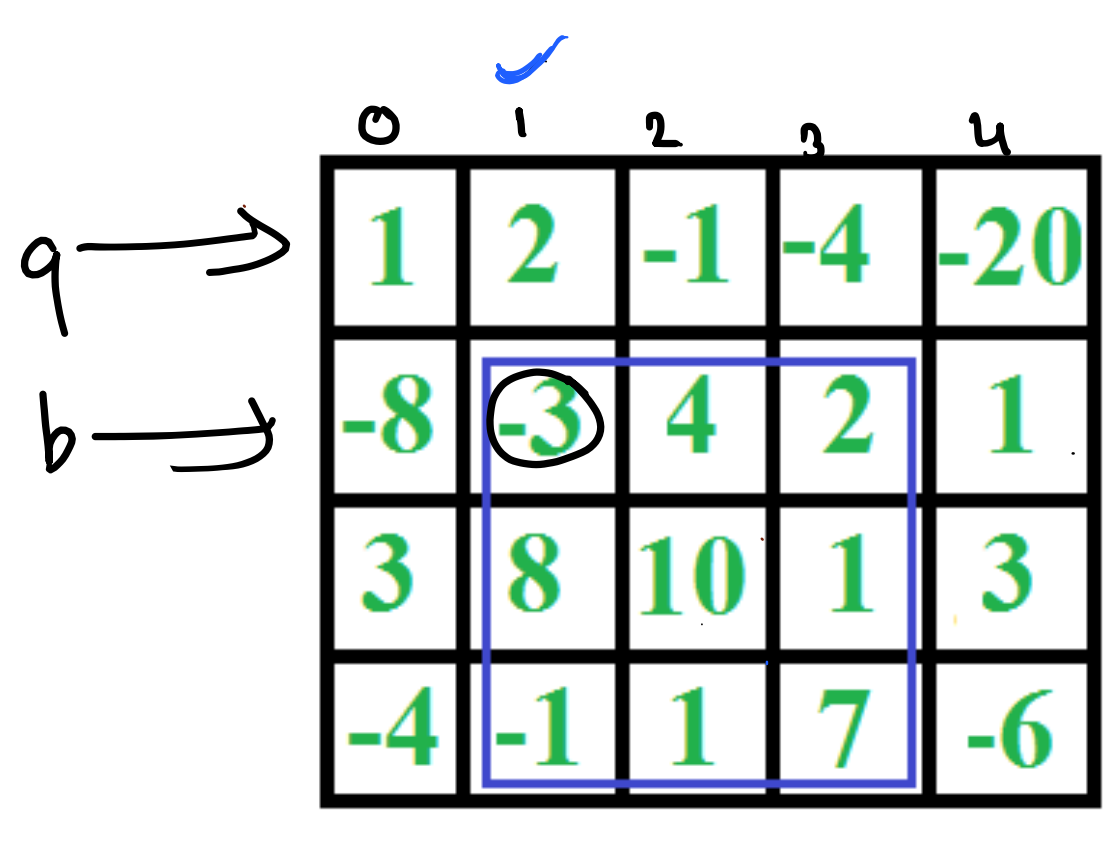


m o s 3

```
6
7 9
0 10
4 5
8 9
4 10
5 7
```







	0	1	2	3	4
a →	1	2	-1	-4	-20
b →	-8	-3	4	2	1
	3	8	10	1	3
	-4	-1	1	7	-6

-8 -3 4 2 1
-4 -1 1 7 -6
1 2 -1 -4 -20
3 8 10 13